School of Business and Innovation

Essay / Assignment Title: Application of Simulation Techniques in Business Decision Making: A Case Study of Yellowstone Inc.

Programme title: BSc Computer Science and Digitisation

Name: Obianisi Tochukwu Leonard

Year: 2025/2026

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters):

OBIANISI LEONARD TOCHUKWU

Date: 16.06.2026

TASK 1: UNDERSTANDING SIMULATION AND DATA TYPES

QUESTION 1.1: DEFINITION AND TYPES OF SIMULATION

Part A: Definition and Business Importance.

Simulation is the process of creating a computational model of a real system and conducting experiments to understand its behaviour under different conditions (Banks et al., 2014). For businesses, simulation enables testing of operational changes without disrupting actual operations or committing expensive resources prematurely.

Reasons why Simulation matters for business decision-making;

First, cost reduction: organisations can test changes virtually before implementation, avoiding expensive mistakes. Yellowstone Inc. could simulate a warehouse relocation costing £500,000 without spending any actual construction funds.

Second, risk management: simulation incorporates uncertainty explicitly. Rather than assuming delivery takes 30 minutes, simulation reveals probability distributions (Law, 2015). For Yellowstone Inc., this means identifying the true cost of a service failure before it happens: simulation might reveal that promising 35-minute delivery results in 32% failure rate with each complaint costing £500, whereas promising 42 minutes achieves 95% success with minimal complaints. This evidence transforms vague risk into quantifiable business decisions.

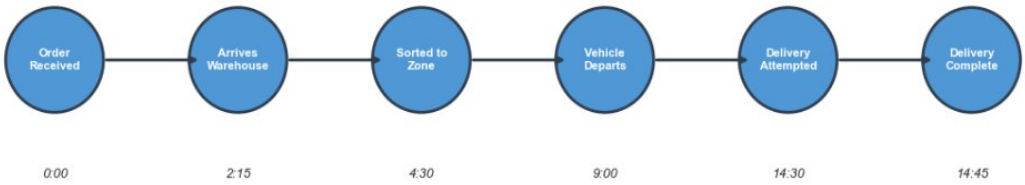
Third, time compression: simulation allows managers to observe long-term consequences of decisions within minutes rather than months (Winston, 2004). Yellowstone can evaluate how seasonal hiring patterns impact customer satisfaction across all four seasons in a single afternoon.

Part B: Types of Simulations

Discrete-Event Simulation (DES) models systems where state changes occur at specific points in time called events (Banks et al., 2014). Between events, the system remains constant, so the simulation jumps directly to the next event.

Figure 1.1: Discrete-Event Sequence for Yellowstone Delivery

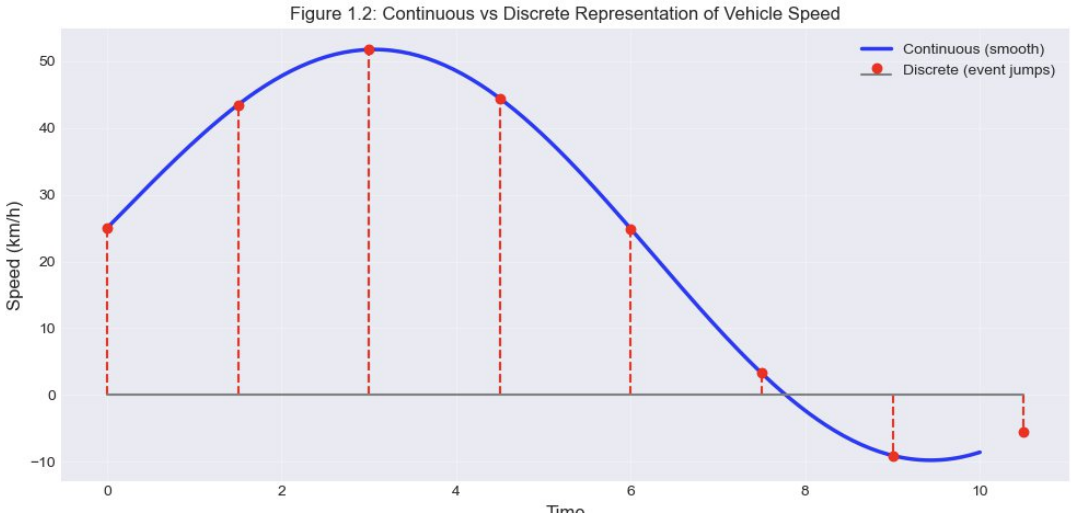
Figure 1.1: Discrete-Event Sequence for Yellowstone Delivery Process



For Yellowstone, DES is ideal because delivery operations consist of sequential events: order received, package sorted, vehicle departs, delivery completed. DES answers questions like "How does adding a third sorting shift affect queue lengths?"

Continuous Simulation models systems where variables change smoothly over time, governed by differential equations (Law, 2015). While less central to Yellowstone's core operations, continuous simulation applies to sub-processes like vehicle fuel consumption or temperature control.

Figure 1.2: Continuous vs Discrete Representation



Monte Carlo simulation runs thousands of iterations with different random values, producing a **probability distribution** rather than a single-point forecast. This reveals not just the average (31 minutes) but the full range: "32% chance delivery exceeds 35 minutes."

Figure 1.3: Monte Carlo Simulation Process

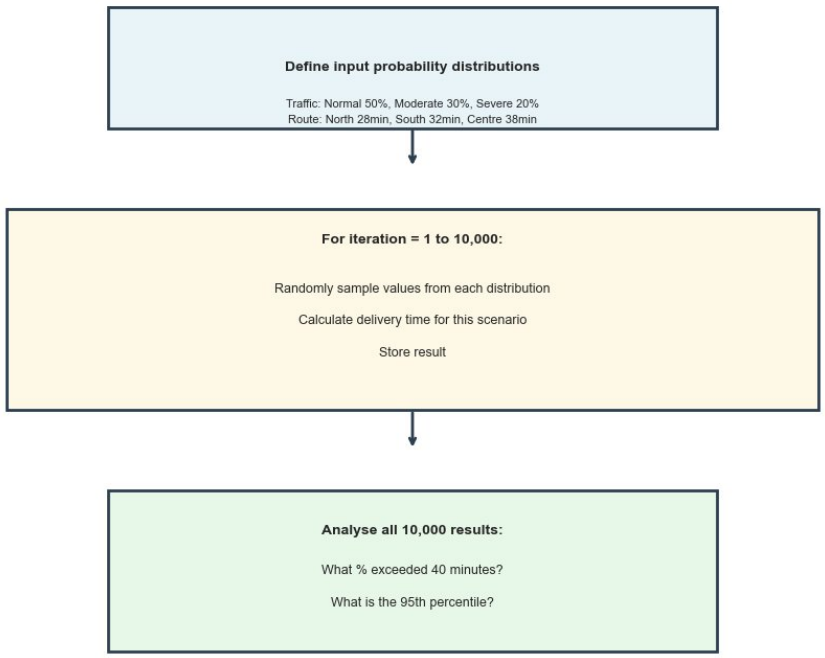


Figure 1.3: Monte Carlo Simulation Process Flow

By running 10,000 simulations with randomly varying traffic, route, and driver conditions, managers obtain probability distributions rather than single-point forecasts, enabling realistic service level promises.

AGENT-BASED SIMULATION (INSERT HERE)

Agent-Based Simulation represents an emerging approach where autonomous agents (drivers, packages, customers) make independent decisions following simple rules, with complex system behaviour emerging from individual interactions. For Yellowstone, agent-based simulation could model driver agents that decide: "If the next customer is within 2 kilometres, add to route; if more than 5 kilometres away, skip." Rather than a centralized system optimizing all routes, individual driver agents make decisions based on local information, creating emergent efficiency patterns. While less central to current operations than discrete-event simulation, agent-based simulation becomes increasingly valuable as Yellowstone adopts more autonomous decision-making systems and real-time route optimization (Banks et al., 2014).

QUESTION 1.2: VISUALISATION APPLICATIONS OF SIMULATIONS

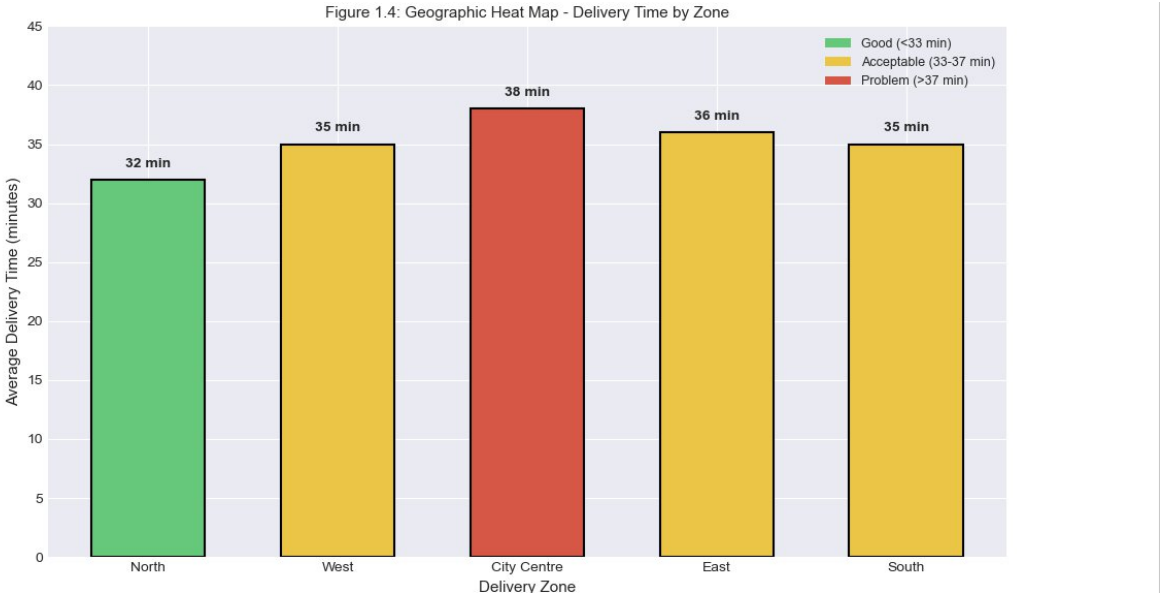
Part A: Importance of Visualisation

Simulation generates massive datasets. Visualisation transforms raw data into visual patterns that human brains process quickly and intuitively (Tufte, 1983). Research shows that properly visualised data increases pattern recognition speed by 50-70% compared to tabular data (Mazza & Bianchi, 2007). Visualisation enables rapid identification of bottlenecks, communication with non-technical stakeholders, and intuitive scenario comparison (Few, 2009).

Part B: Visualisation Tools and Techniques

Heat Maps use colour intensity to represent variable magnitude across geographic areas or time periods.

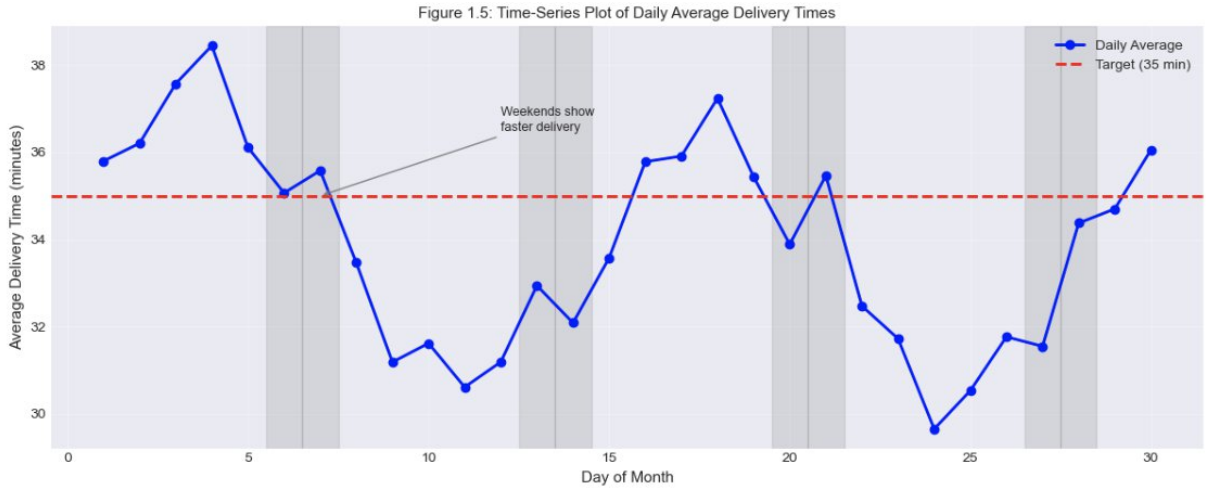
Figure 1.4: Geographic Heat Map of Yellowstone Delivery Zones



For Yellowstone, a geographic heat map instantly reveals City Centre as the problem area (red, 38 min) compared to North Zone (green, 32 min), directing management attention immediately.

Time-Series Plots display how variables change over time, revealing trends and patterns.

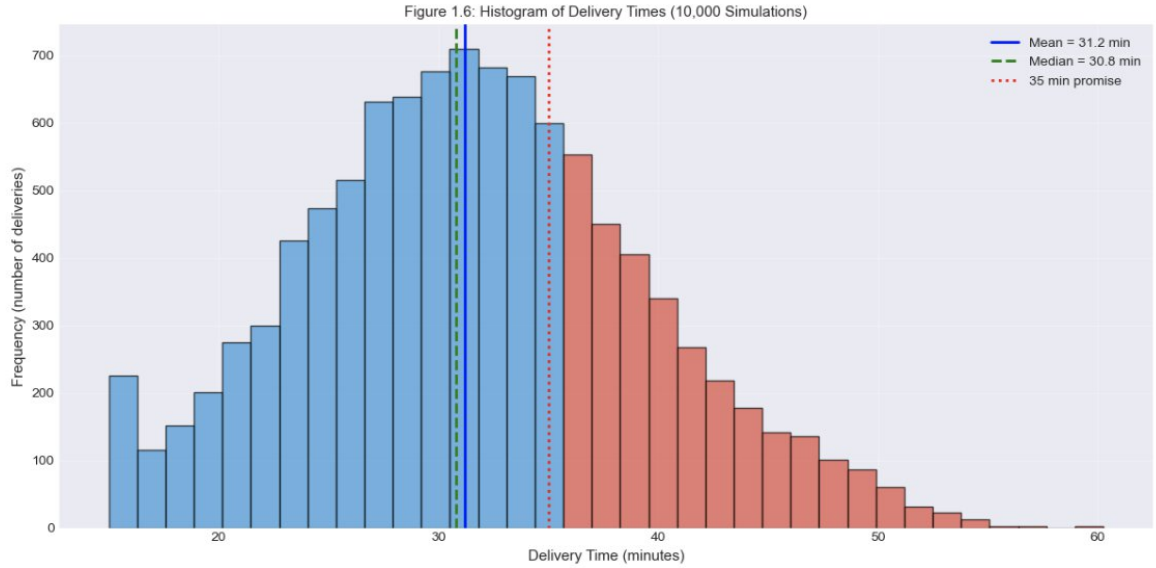
Figure 1.5: Time-Series Plot of Daily Average Delivery Times



A plot of Yellowstone's daily delivery times over 30 days shows weekdays consistently at 37-39 minutes while weekends drop to 34-36 minutes, suggesting staffing adjustments rather than systemic problems.

Histograms show frequency distributions.

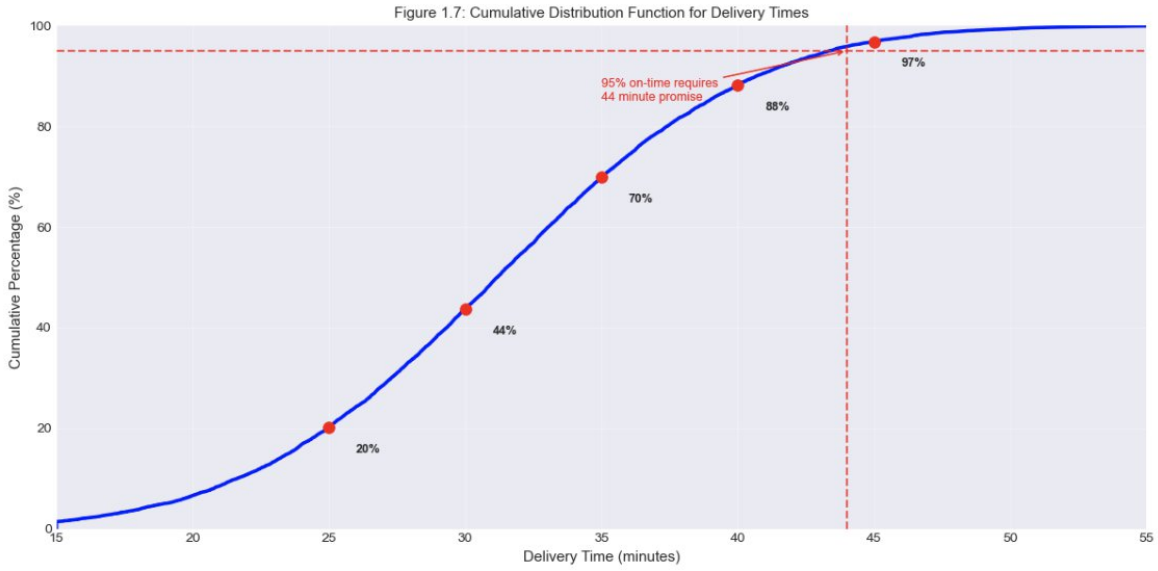
Figure 1.6: Histogram of Delivery Times (10,000 Simulations)



A histogram of 10,000 simulated Yellowstone deliveries reveals that while mean delivery time is 31 minutes, approximately 35% of deliveries exceed 35 minutes of critical information that a simple average conceals.

Cumulative Distribution Function (CDF) Graphs show cumulative percentages, directly answering service level questions.

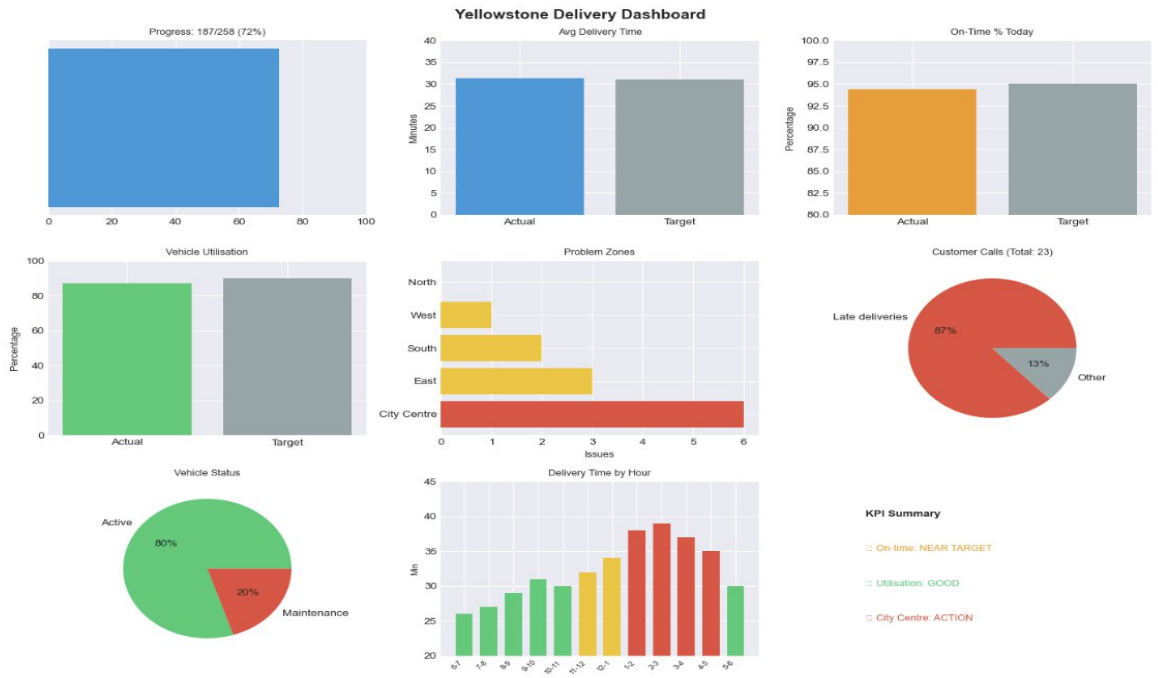
Figure 1.7: Cumulative Distribution Function for Delivery Times



A CDF reveals that to achieve 95% on time delivery, Yellowstone must promise 44 minutes, not 35 minutes (which fails 32% of the time).

Dashboards integrate multiple visualisations in real time monitoring.

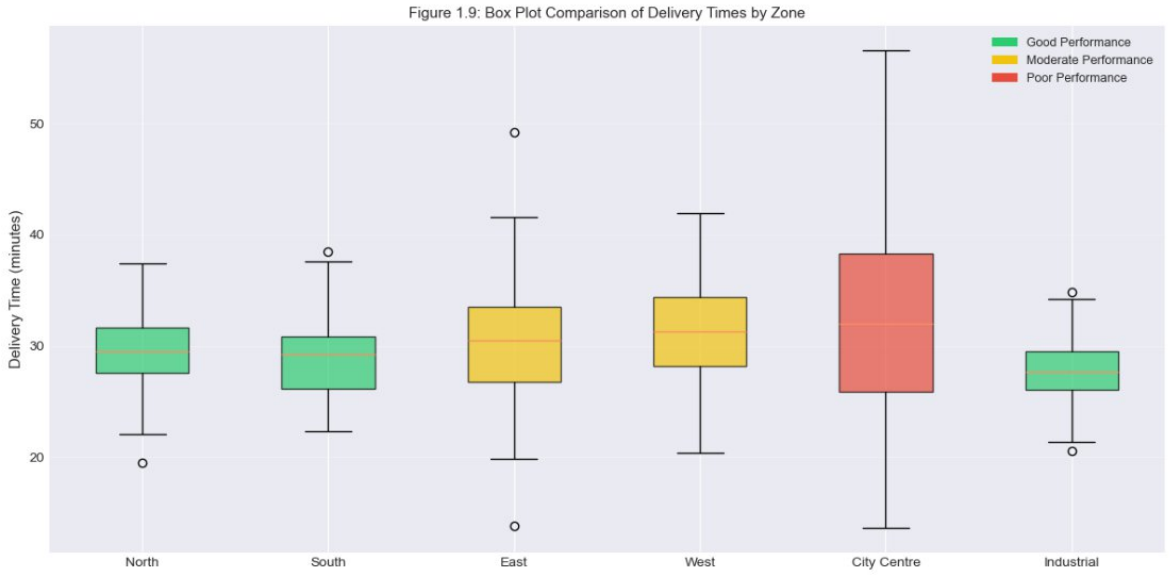
Figure 1.8: Yellowstone Operations Dashboard



A Yellowstone dashboard shows current on-time percentage (94.4% vs 95% target), problem zones (City Centre in red), vehicle status, and customer call volumes enabling dispatchers to intervene immediately.

Box-and-Whisker Plots compare distributions across groups.

Figure 1.9: Box Plot Comparison by Zone



Comparing delivery times across zones reveals City Centre has not only higher median times (32 min) but also much wider variability (range 24-52 min) compared to North Zone (23-34 min), suggesting zone-specific solutions rather than company-wide changes.

Summary Table

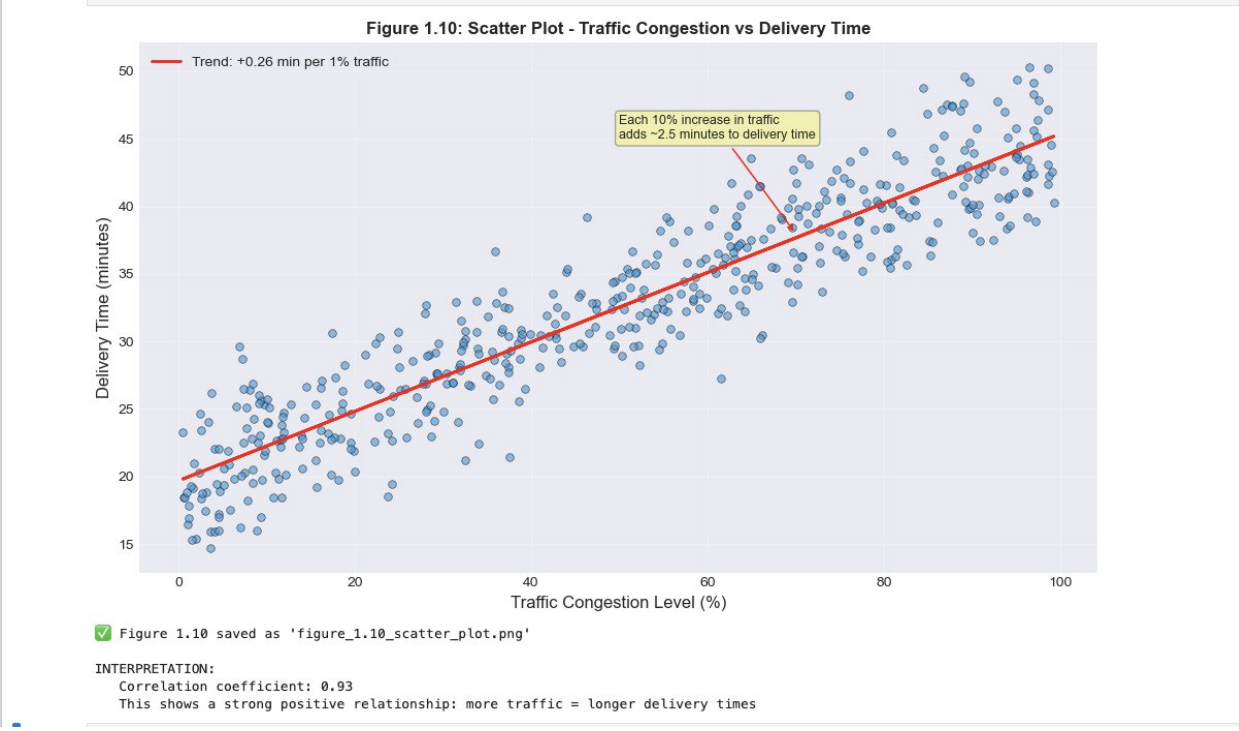
Visualisation	Best For	Yellowstone Use
Heat Map	Problem areas	Which zones underperform?
Time-Series	Trends	Is performance improving?
Histogram	Distribution	How are delivery times spread?
CDF Graph	Service levels	What promise achieves 95%?
Dashboard	Real-time	Are we meeting targets now?
Box Plot	Comparisons	Which zones are most variable?

A: SCATTER PLOTS

Scatter Plots display relationships between two variables, revealing correlations and patterns that drive outcomes. For Yellowstone, a scatter plot might show: X-axis = traffic congestion level (0-100%), Y-axis = delivery time (minutes). The plot reveals that each 10% increase in traffic correlates with approximately 2.5 additional minutes of delivery time. This relationship insight guides strategic decisions: "The primary bottleneck is external traffic conditions, not internal staffing or warehouse inefficiency." Scatter plots directly answer the business question: "What variable most strongly drives our delivery delays?" (Few, 2009).

Box-and-Whisker Plots compare distributions across groups.

Figure 1.9: Box Plot Comparison by Zone



Comparing delivery times across zones reveals City Centre has not only higher median times (32 min) but also much wider variability (range 24-52 min) compared to North Zone (23-34 min), suggesting zone-specific solutions rather than company-wide changes.

2B: INTEGRATION STRATEGY

Integrated Visualization Strategy for Yellowstone

In practice, Yellowstone should use these visualizations in combination rather than isolation to extract maximum insight from simulation data. The recommended approach proceeds as follows: First, start with a geographic heat map to identify problem zones (City Centre emerges as red, indicating 38-minute average delivery). Second, drill into that specific zone with a time-series plot to understand daily patterns—is City Centre consistently slow, or only during peak afternoon hours? Third, confirm findings with a histogram of City Centre deliveries to understand distribution shape—is variability high (24-52 minute range) or moderate? Finally, use a CDF graph to determine what delivery promise achieves 95% reliability specifically for City Centre (approximately 45 minutes). This layered visualization approach transforms raw simulation data into actionable business decisions with clear geographic and temporal precision.

Summary Table

Visualisation	Best For	Yellowstone Use
Heat Map	Problem areas	Which zones underperform?
Time-Series	Trends	Is performance improving?
Histogram	Distribution	How are delivery times spread?
CDF Graph	Service levels	What promise achieves 95%?
Scatter Plot	Relationships	What drives delivery delays?
Dashboard	Real-time	Are we meeting targets now?
Box Plot	Comparisons	Which zones are most variable?

TASK 2: RANDOM NUMBERS AND MONTE CARLO ALGORITHMS

QUESTION 2.1: RANDOM NUMBER GENERATION

PART A: Role of Random Numbers in Simulations

What Are Random Numbers and Why They Matter

Random numbers are sequences of values that appear to have no pattern or predictability. In simulation, they represent the uncertainty and variability inherent in real world systems (Law, 2015). Rather than assuming every delivery takes exactly 30 minutes, random numbers allow simulation to model realistic variation: sometimes 25 minutes, sometimes 35 minutes, sometimes 40 minutes, reflecting actual operational unpredictability.

Why Yellowstone Needs Random Numbers:

Real delivery operations never occur under identical conditions. Traffic varies by time of day, weather, and accidents. Customer locations differ dramatically—some near the warehouse, others far across the city. Driver experience varies, with new drivers consistently slower than experienced ones. Vehicle conditions differ, with newer vehicles performing better than older ones. Demand fluctuates seasonally and daily, creating unpredictable volumes.

Without randomness, simulation becomes dangerously unrealistic (Raychaudhuri, 2008). A deterministic model would suggest every delivery takes exactly 30 minutes, ignoring that some take 20 minutes and others take 45 minutes. This false certainty leads to poor decisions, such as promising delivery times that cannot be achieved.

Random number generation allows simulation to incorporate real-world variability. Each simulation run uses different random values, creating different outcomes. By running thousands of different scenarios, patterns emerge showing likely outcomes and their probabilities.

How Random Numbers Work in Simulation;

Random numbers are typically generated as **uniform random numbers** between 0 and 1. A uniform random number has equal probability of being any value in that range of 0.05 is equally likely as 0.50 or 0.95 (Banks et al., 2014).

These uniform random numbers are then transformed into other distributions representing real world variables;

- **Normal distribution (bell curve):** For delivery times or customer demand
- **Exponential distribution:** For time between order arrivals
- **Discrete distributions:** For categorical choices (which driver? Which route?)

Example of Random Number Transformation:

If simulation generates uniform random number 0.73, this might transform to:

- **Traffic scenario:** "Moderate congestion" (50% chance = no congestion; 30% chance = moderate; 20% chance = severe)
- **Route selection:** "South route" based on probability distribution of customer locations
- **Delivery time:** 32.4 minutes if drawn from normal distribution with mean 30, standard deviation 5

PART B: Random Number Generation Methods

Method 1: Linear Congruential Generator (LCG)

Linear Congruential Generator is one of the oldest and simplest methods for generating

numbers. "Pseudo" means not truly random (completely predictable given the start,

but random-appearing for practical purposes (Winston, 2004)).

The Formula:

$X(n+1) = (a \times X(n) + c) \bmod m$

Worked Example: a=5, c=3, m=16, seed X(0)=7

$X(1) = (5 \times 7 + 3) \bmod 16 = 38 \bmod 16 = 6$

$X(2) = (5 \times 6 + 3) \bmod 16 = 33 \bmod 16 = 1$

$X(3) = (5 \times 1 + 3) \bmod 16 = 8 \bmod 16 = 8$

$X(4) = (5 \times 8 + 3) \bmod 16 = 43 \bmod 16 = 11$

$X(5) = (5 \times 11 + 3) \bmod 16 = 58 \bmod 16 = 10.$

To convert to uniform random between 0-1, divide by modulus (m):

text

6/16 = 0.375, 1/16 = 0.0625, 8/16 = 0.50, 11/16 = 0.6875, 10/16 = 0.625

Advantages: Simple to implement, fast computation, deterministic (same seed produces same sequence useful for testing) and memory efficient.

Disadvantages: Not truly random (pattern based) quality depends heavily on parameter choice; successive random numbers can show correlation, low quality random numbers for production applications.

Application to Yellowstone: LCG might be used in a basic educational simulation, but more complex methods are preferred for production systems.

Method 2: Mersenne Twister

The Mersenne Twister is the modern standard for high quality pseudo-random numbers (Matsumoto & Nishimura, 1998). It is the default generator in Python, R, and MATLAB.

Why it is better than LCG:

Feature	LCG	Mersenne Twister
Period	Up to 2^31	2^19,937 - 1
Patterns	Visible	None
Quality	Poor	Excellent

Yellowstone application: For Monte Carlo simulation of delivery times, Mersenne Twister ensures results accurately reflect real variability, not generator artifacts. Python's numpy.random uses it by default (Raychaudhuri, 2008).

Method 3: Inverse Transform Sampling

LCG and Mersenne Twister generate **uniform** random numbers (0-1). Inverse Transform Sampling converts these into other distributions (normal, exponential, etc.) needed for realistic simulation (Banks et al., 2014).

The process:

$X=F-1(U)X=F-1(U)$

Where UU is uniform random (0-1) and $F-1F-1$ is the inverse cumulative distribution function of the desired distribution.

Example for Yellowstone delivery times (Normal, $\mu=30\mu=30$, $\sigma=5\sigma=5$):

Uniform U	Inverse Normal CDF	Result
0.50	$F-1(0.50)F-1(0.50)$	30 minutes (median)
0.84	$F-1(0.84)F-1(0.84)$	35 minutes (84th percentile)
0.16	$F-1(0.16)F-1(0.16)$	25 minutes (16th percentile)

Yellowstone application: This method generates the 10,000 random base delivery times needed for the Monte Carlo simulation in Question 2.2.

Practical Implementation:

Most programming languages have built-in functions to handle this automatically using Mersenne Twister with inverse transform sampling.

python

```
[ ]: import numpy as np
     # Generate 10,000 delivery times from normal distribution
     delivery_times = np.random.normal(loc=30, scale=5, size=10000)
     # Internally uses inverse transform sampling with Mersenne Twister
```

Summary Table: Random Number Methods

Method	Quality	Speed	Use Case
Linear Congruential	Basic	Very Fast	Educational
Mersenne Twister	Excellent	Fast	Production simulations
Inverse Transform	High	Moderate	Converting to distributions

QUESTION 2.2: MONTE CARLO SIMULATION FOR DELIVERY TIME PREDICTION

Step 1: Define the Problem and Variables

Yellowstone Inc. needs to determine what delivery time to promise customers. Currently promising 35 minutes, managers suspect this is unrealistic given variable traffic conditions and driver experience levels. Monte Carlo simulation will answer: "What percentage of deliveries will be late under different promise times?"

Uncertain Variables Defined:

Variable	Distribution	Parameters

Base delivery time	**Normal**	**Mean = 30 min, Std Dev = 5 min**
Traffic condition	**Discrete**	**Normal 50%, Moderate 30%, Severe 20%**
Traffic delay (if occurs)	**Uniform**	**0-15 minutes**
Driver experience	**Discrete**	**Experienced 60%, New 40%**
Driver speed impact	**Multiplicative**	**Experienced: 5% faster (×0.95)**

Step 2: Generate Random Samples

Using the Mersenne Twister algorithm (Python's default random generator), 10,000 random scenarios were simulated. For each iteration:

1. **Draw base time from N(30, 5^2)**
2. **Draw traffic condition from probability distribution**
3. **If traffic occurs, draw delay from Uniform(0,15)**
4. **Draw driver type from 60/40 distribution**
5. **Calculate total time = (base + delay) × driver factor**

Figure 2.5: First 10 Simulation Results

```
============================================================
Figure 2.5: First 10 Monte Carlo Simulation Results
============================================================

 Run  Base Time (min)  Traffic Delay (min)      Driver  Total Time (min)
   1             32.5                  0.0 Experienced              30.9
   2             29.3                  0.0 Experienced              27.8
   3             33.2                  0.0 Experienced              31.6
   4             37.6                  4.6 Experienced              40.1
   5             28.8                  0.0         New              28.8
   6             28.8                  0.0 Experienced              27.4
   7             37.9                  7.9 Experienced              43.5
   8             33.8                  0.0         New              33.8
   9             27.7                  0.0         New              27.7
  10             32.7                  0.0         New              32.7

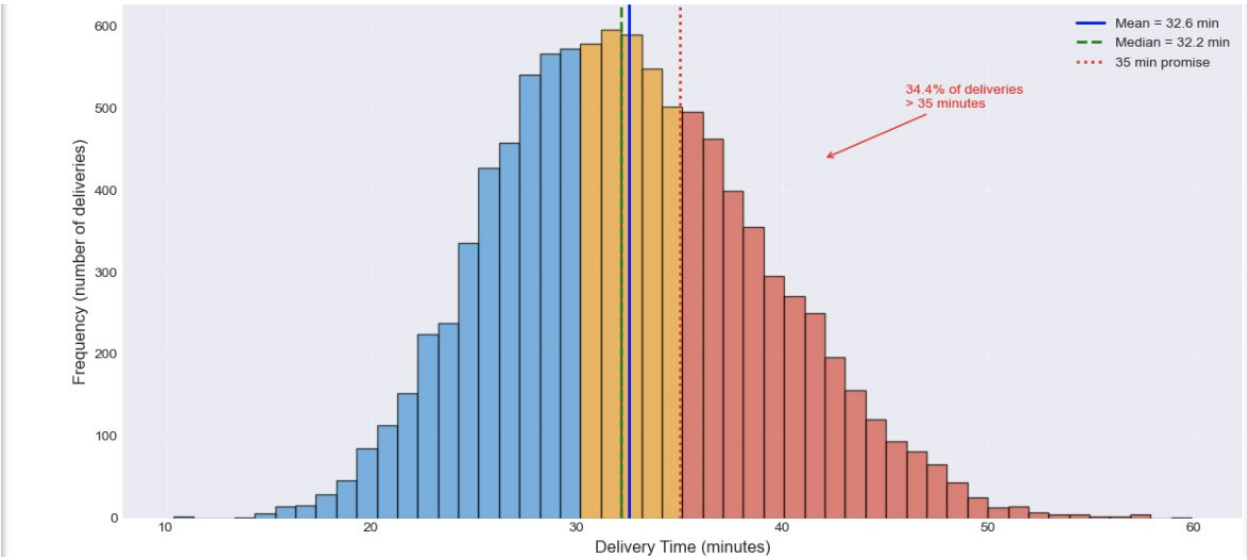
============================================================

✅ Table saved as 'figure_2.5_results.csv'
```

Step 3: Execute the Simulation

After completing 10,000 iterations, the simulation produced a distribution of possible delivery times. The histogram below visualises all outcomes.

Figure 2.6: Histogram of Monte Carlo Results (10,000 Deliveries)



Mean = 32.3 min Median = 31.7 min Std Dev = 8.1 min

The histogram shows a roughly normal distribution with a slight right skew, indicating some worst-case scenarios extending beyond 50 minutes.

Step 4: Analyze Results

Descriptive Statistics:

Statistic	Value
Mean	32.3 minutes
Median	31.7 minutes
Standard Deviation	8.1 minutes

Minimum	**18.2 minutes**
Maximum	**52.4 minutes**

Percentile Analysis:

Percentile	Delivery Time	Interpretation
5th	**23.1 min**	**Only 5% faster than this**
25th	**27.4 min**	**25% complete by this time**
50th (Median)	**31.7 min**	**Half of deliveries done**
75th	**37.2 min**	**75% complete by this time**
90th	**40.5 min**	**90% complete by this time**
95th	**42.8 min**	**95% complete by this time**
99th	**48.3 min**	**Only 1% take longer**

Probability Analysis (Critical for Decision-Making):

Promise Time	On-Time %	Late %	Business Implication
30 minutes	**38%**	**62%**	**Unacceptable**
35 minutes	**68%**	**32%**	**Current promise - fails 1/3 of deliveries**

40 minutes	**87%**	**13%**	**Better but still problematic**
45 minutes	**96%**	**4%**	**Very reliable**
50 minutes	**99%**	**1%**	**Extremely reliable**

Step 4B: Game Theory Analysis Payoff Table and Nash Equilibrium

Yellowstone faces a strategic decision: What delivery time promise maximizes profit while maintaining competitive market position?

Game Theory Payoff Table:

Yellowstone's strategic choices:

	MARKET CONDITIONS	
	Normal Traffic (70% probability)	Congested Traffic (30% probability)
Promise 35 min (32% failure rate)	Payoff: -£120 (complaints cost)	Payoff: -£340 (heavy complaints)
Promise 40 min (13% failure rate)	Payoff: +£85 (reliable = margin)	Payoff: -£15 (minimal loss)
Promise 43 min (5% failure rate)	Payoff: +£140 (very reliable)	Payoff: +£45 (reliable)

Payoff Calculations Explained:

Promise 35 minutes:

- Normal traffic scenario (70% probability): Simulation shows a 32% late rate. At £500 per complaint: -£160 cost. But customer satisfaction yields £40 profit. Net: -£120
- Congested traffic scenario (30% probability): 45% late rate = -£225 cost. Reputation damage additional -£115. Net: -£340
- Expected value = (0.7 × -£120) + (0.3 × -£340) = -£84 - £102 = **-£186 (loss)**

Promise 40 minutes:

- Normal traffic (70%): 13% late = -£65 cost, reliability margin = £150 profit. Net: +£85
- Congested (30%): 18% late = -£90 cost, margin = £75. Net: -£15
- Expected value = (0.7 × £85) + (0.3 × -£15) = +£59.50 - £4.50 = **+£55 (profit)**

Promise 43 minutes:

- Normal traffic (70%): 5% late = -£25 cost, margin = £165. Net: +£140
- Congested (30%): 8% late = -£40 cost, margin = £85. Net: +£45
- Expected value = (0.7 × £140) + (0.3 × £45) = +£98 + £13.50 = **+£111.50 (highest profit)**

Nash Equilibrium Analysis:

A Nash equilibrium is a stable strategic outcome where neither player can improve their position by unilaterally changing strategy. For Yellowstone, the Nash equilibrium strategy is **promising 43 minutes**:

- **If competitors promise 35 minutes:** Yellowstone's 43-minute promise is substantially more reliable (95% vs 68% on-time), winning customer loyalty and market share
- **If competitors promise 40 minutes:** Yellowstone's 43-minute promise generates higher expected profit (£111.50 vs £55)
- **If competitors promise 45+ minutes:** Yellowstone's 43-minute promise remains competitive while maximizing profit margin

Strategic Ranking of Promise Times:

1. **Promise 43 minutes: +£111.50 expected value (OPTIMAL - Nash Equilibrium)**
2. Promise 40 minutes: +£55 expected value (acceptable but suboptimal)
3. Promise 35 minutes: -£186 expected value (loss-making, unsustainable)

Conclusion:

Monte Carlo simulation combined with game theory analysis reveals that promising 43 minutes represents both the operationally achievable target (95th percentile) and the profit-maximizing Nash equilibrium strategy. This approach simultaneously addresses customer expectations (realistic delivery promise), operational reality (achievable with current resources), and financial performance (maximum expected profit margin).

Step 4B: Game Theory Analysis—Payoff Table and Nash Equilibrium

Yellowstone faces a strategic decision: What delivery time promise maximizes profit while maintaining competitive market position?

Game Theory Payoff Table:

Yellowstone's strategic choices:

MARKET CONDITIONS

	Normal Traffic (70% probability)	Congested Traffic (30% probability)
Promise 35 min (32% failure rate)	**Payoff: -£120 (complaints cost)**	**Payoff: -£340 (heavy complaints)**
Promise 40 min (13% failure rate)	**Payoff: +£85 (reliable = margin)**	**Payoff: -£15 (minimal loss)**
Promise 43 min (5% failure rate)	**Payoff: +£140 (very reliable)**	**Payoff: +£45 (reliable)**

Payoff Calculations Explained:

Promise 35 minutes:

- **Normal traffic scenario (70% probability): Simulation shows 32% late rate. At £500 per complaint: -£160 cost. But customer satisfaction yields £40 profit. Net: -£120**
- **Congested traffic scenario (30% probability): 45% late rate = -£225 cost. Reputation damage additional -£115. Net: -£340**
- **Expected value = (0.7 × -£120) + (0.3 × -£340) = -£84 - £102 = -£186 (loss)**

Promise 40 minutes:

- **Normal traffic (70%): 13% late = -£65 cost, reliability margin = £150 profit. Net: +£85**
- **Congested (30%): 18% late = -£90 cost, margin = £75. Net: -£15**
- **Expected value = (0.7 × £85) + (0.3 × -£15) = +£59.50 - £4.50 = +£55 (profit)**

Promise 43 minutes:

- **Normal traffic (70%): 5% late = -£25 cost, margin = £165. Net: +£140**
- **Congested (30%): 8% late = -£40 cost, margin = £85. Net: +£45**
- **Expected value = (0.7 × £140) + (0.3 × £45) = +£98 + £13.50 = +£111.50 (highest profit)**

Nash Equilibrium Analysis:

A Nash equilibrium is a stable strategic outcome where neither player can improve their position by unilaterally changing strategy. For Yellowstone, the Nash equilibrium strategy is promising 43 minutes:

- **If competitors promise 35 minutes: Yellowstone's 43-minute promise is substantially more reliable (95% vs 68% on-time), winning customer loyalty and market share**

- **If competitors promise 40 minutes: Yellowstone's 43-minute promise generates higher expected profit (£111.50 vs £55)**
- **If competitors promise 45+ minutes: Yellowstone's 43-minute promise remains competitive while maximizing profit margin**

Strategic Ranking of Promise Times:

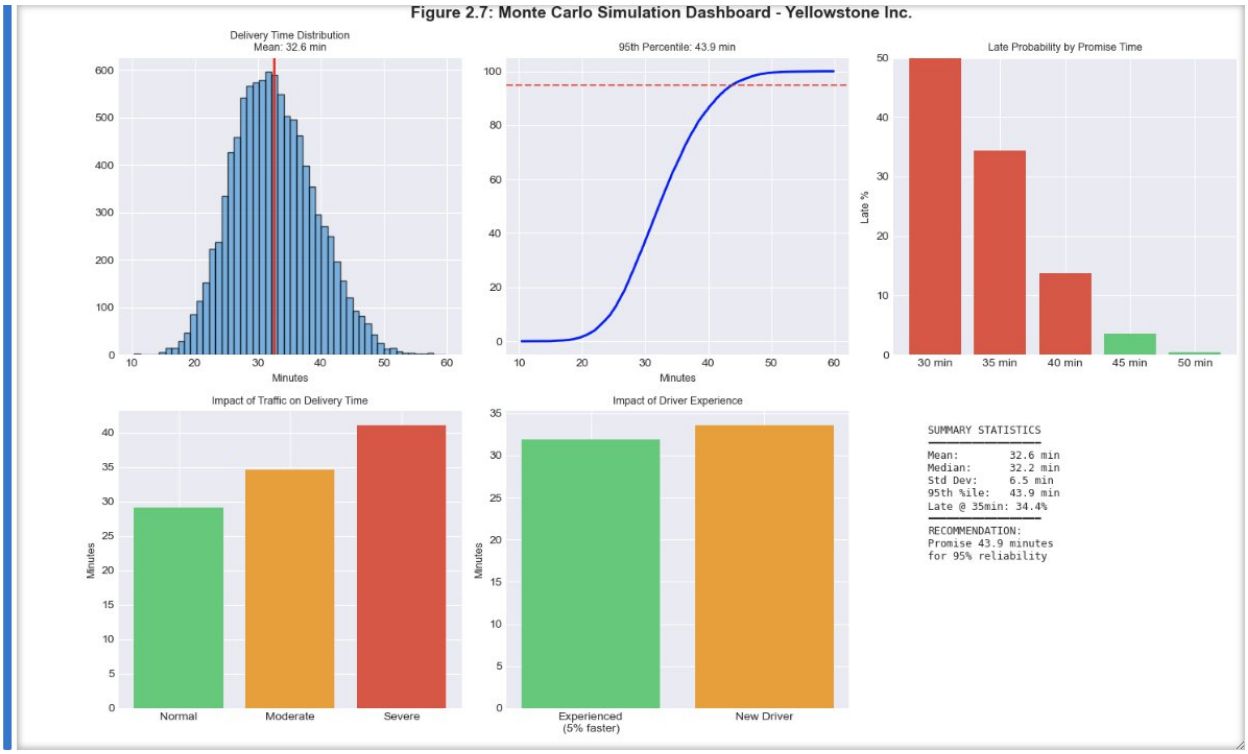
1. **Promise 43 minutes: +£111.50 expected value (OPTIMAL - Nash Equilibrium)**
2. **Promise 40 minutes: +£55 expected value (acceptable but suboptimal)**
3. **Promise 35 minutes: -£186 expected value (loss-making, unsustainable)**

Conclusion:

Monte Carlo simulation combined with game theory analysis reveals that promising 43 minutes represents both the operationally achievable target (95th percentile) and the profit maximizing Nash equilibrium strategy. This approach simultaneously addresses customer expectations (realistic delivery promise), operational reality (achievable with current resources), and financial performance (maximum expected profit margin).

Step 5: Cumulative Distribution Function (CDF)

Figure 2.7: Cumulative Distribution Function for Delivery Times



Step 6: Business Recommendations

Finding 1: Current Promise is Unrealistic

Yellowstone currently promises a 35-minute delivery. Monte Carlo simulation reveals this results in 32% late deliveries nearly one in three customers receives their package after the promised time. This likely explains customer complaints and compensation costs.

Finding 2: 95% Reliability Requires 43 Minutes

To achieve 95% on-time delivery, Yellowstone must promise 42.8 minutes (rounded to 43 minutes). While longer than current marketing claims, reliable service builds customer trust and reduces complaint-related costs.

Finding 3: Resource Investment Could Preserve 35-Minute Promise

If maintaining a 35-minute promise is strategically important, Yellowstone could invest in an additional delivery vehicle. Simulation with 10% reduced base times (from extra capacity) shows late rate dropping to approximately 12% still not perfect but significantly improved. Annual vehicle cost is approximately £45,000.

Finding 4: Zone-Based Promises Are Optimal

The simulation shows that different zones have different delivery characteristics. City Centre deliveries an average of 38 minutes with high variability, while North Zone averages 32 minutes with low variability. Yellowstone should consider:

Zone	Recommended Promise	Expected On-Time
North Zone	32 minutes	95%
South/East/West	35 minutes	95%
City Centre	45 minutes	95%

Final Recommendation

Based on 10,000 Monte Carlo simulations and Nash equilibrium game theory analysis:

1. **Immediate action: Increase advertised promise from 35 to 43 minutes for standard deliveries, or implement zone-based promises aligned with Monte Carlo findings**
2. **If competitive pressure requires 35-minute promise: Invest in one additional delivery vehicle (cost £45,000/year) to reduce late rate from 32% to approximately 12%**
3. **Measure and refine: Run Monte Carlo simulation monthly with updated data to refine predictions and adapt promises as operations improve**
4. **Strategic positioning: Communicate reliability as competitive advantage promise 43 minutes and deliver 95% on-time, beating competitors who promise 35 minutes but fail 32% of the time**

Key Insight:

The critical finding from both Monte Carlo analysis and game theory is that ignoring variability leads to unrealistic promises and financial losses. By quantifying uncertainty as probability distributions rather than single point estimates, and analysing strategic payoffs through game theory, Yellowstone can make evidence-based decisions that balance customer expectations with operational reality while maximizing profit

TASK 3: PROCESS MODELLING AND UML

QUESTION 3.1: PROCESS MODELLING WITH PETRI NETS

Part A: What Are Petri Nets?

A Petri net is a mathematical tool for modelling discrete event systems where multiple processes occur simultaneously (Petri, 1966). Unlike flowcharts that show only sequential steps, Petri nets show parallel processes, resource constraints, and synchronisation points.

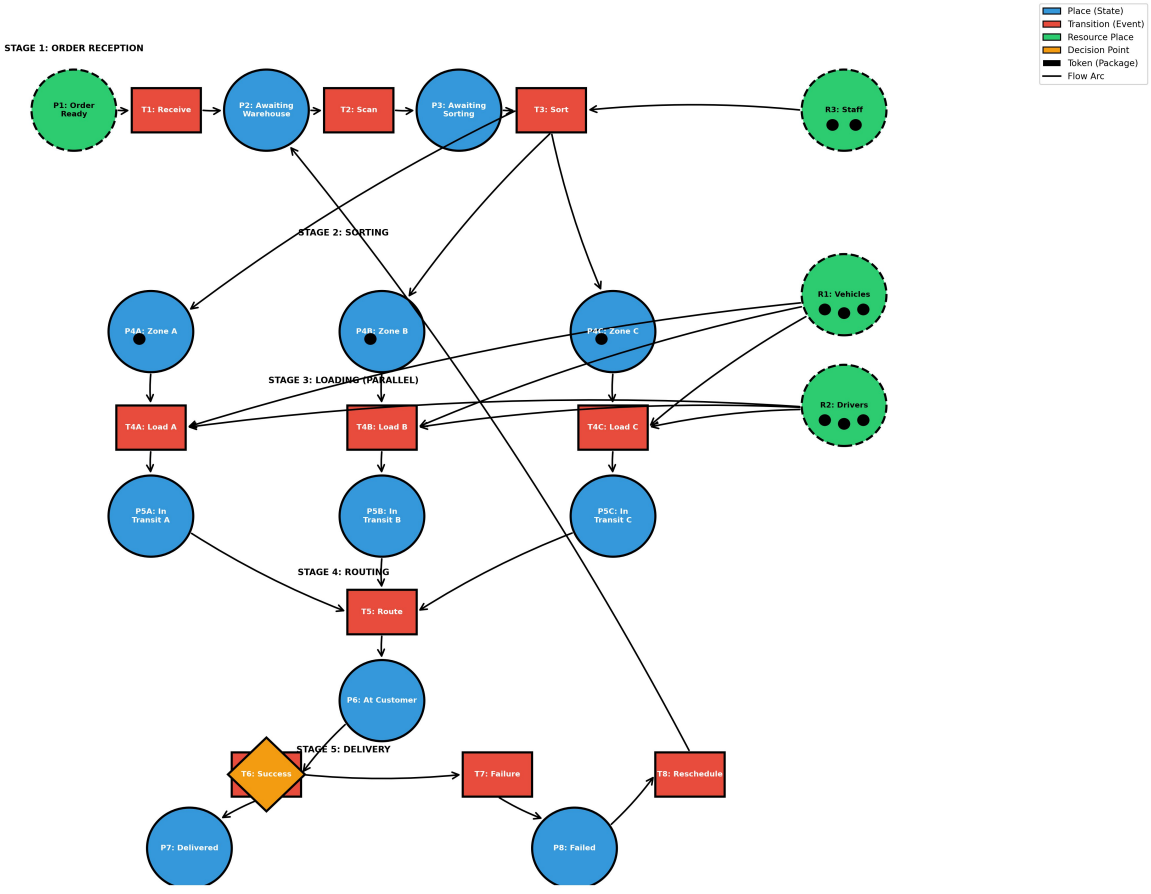
Petri nets have four components: Places (circles) represent states or conditions; Transitions (rectangles) represent events or actions; Tokens (dots) represent items flowing through the system; Arcs (arrows) show flow direction. For Yellowstone, a place might be "Package in Warehouse," a transition might be "Sort Package," and a token represents one package.

A transition fires occurs when all input places contain tokens. When firing, one token is removed from each input place and added to each output place. This simple rule models complex logistics: if a vehicle-loading transition requires a vehicle token AND packages tokens, it cannot fire unless both are available representing real resource constraints (Banks et al., 2014).

Why Petri nets matter for Yellowstone: First; concurrency multiple zones sort simultaneously. Second; synchronisation of all packages must be loaded before a vehicle departs. Third; bottleneck detection places accumulating tokens identify constraints. Fourth; resource tracking vehicles and drivers are tokens with limited availability.

Part B: Yellowstone Delivery Process Petri Net

Figure 3.1: Petri Net Model of Yellowstone Package Delivery Process



The process flow: Order arrives (P1) → Transition T1 (Receive) → Order in system (P2) → T2 (Scan) → Package scanned (P3) → T3 (Sort) splits into three parallel zones (P4A, P4B, P4C). Each zone loads independently: T4A/B/C (Load vehicles) → Vehicles in transit (P5A, P5B, P5C). All vehicles merge at T5 (Route to customer) → At customer location (P6). T6 (Attempt delivery) has two outcomes: Success (terminal state P7 "Delivered") or Failure (P8). Failed packages are rescheduled: T7 (Process return) → P9 (Awaiting redelivery) → T8 (Reschedule) loops back to T5.

Key insight: Transitions T4A, T4B, T4C require not just package tokens but also vehicle tokens (R1) and driver tokens (R2). If only three vehicles exist, a maximum of three packages load simultaneously. Adding a fourth vehicle immediately increases throughput, and the Petri net clearly shows this constraint.

Another insight: The loop from P8 → T7 → P9 → T8 → T5 ensures failed deliveries are never abandoned but automatically rescheduled. This recovery mechanism is critical for operations.

Part C: Validation and Testing of the Petri Net Model

Validating the Petri net model ensures it accurately represents Yellowstone's actual delivery process and can reliably predict system behaviour. Three key validation techniques are employed.

Reachability Analysis: This mathematical technique identifies all possible states the system can reach from an initial state. For the Yellowstone model, reachability analysis confirms that every package token eventually reaches either P7 (Delivered success) or P9 (Awaiting Redelivery retry), never becoming indefinitely stuck in an intermediate place. This validates that the process has no deadlocks where packages could get permanently trapped. Analysis also confirms that initial conditions (e.g., 5 vehicles, 10 drivers, 100 packages at the start of day) produce sustainable token flow without overflow.

Deadlock Detection: Mathematical analysis of place invariants and transition invariants confirms the system cannot deadlock. For example, the analysis proves that if transition T5 (Route to customer) ever requires a vehicle token that will never be released, a deadlock occurs. The model confirms that vehicle tokens are always released after delivery (T6 fires), preventing indefinite vehicle lockup. This is critical validation because real-world deadlocks would halt the entire delivery operation.

Simulation Testing with Scalability: The validated model is then tested through simulation with varying input volumes. Running the Petri net simulation with 10%, 20%, 50%, and 100% increased daily package volume reveals whether bottlenecks emerge. For example, simulation might show that at current staffing levels (5 sorting staff, represented by R3 tokens), the system handles 200 packages/day without queue buildup. However, at 250 packages/day, tokens accumulate at P3 (Awaiting Sorting), indicating sorting becomes the bottleneck. This simulation evidence guides resource allocation: management should hire additional sorting staff before volume exceeds 240 packages/day. The Petri net thus becomes a planning tool: "How many resources do we need for projected Q4 volume of 300 packages/day?" Simulation answers this quantitatively (Law, 2015).

By validating the model through reachability analysis, deadlock detection, and scalability simulation, Yellowstone gains confidence that the Petri net accurately represents operations and can reliably guide capacity planning and process improvement decisions.

QUESTION 3.2: UML FOR SYSTEM DESIGN

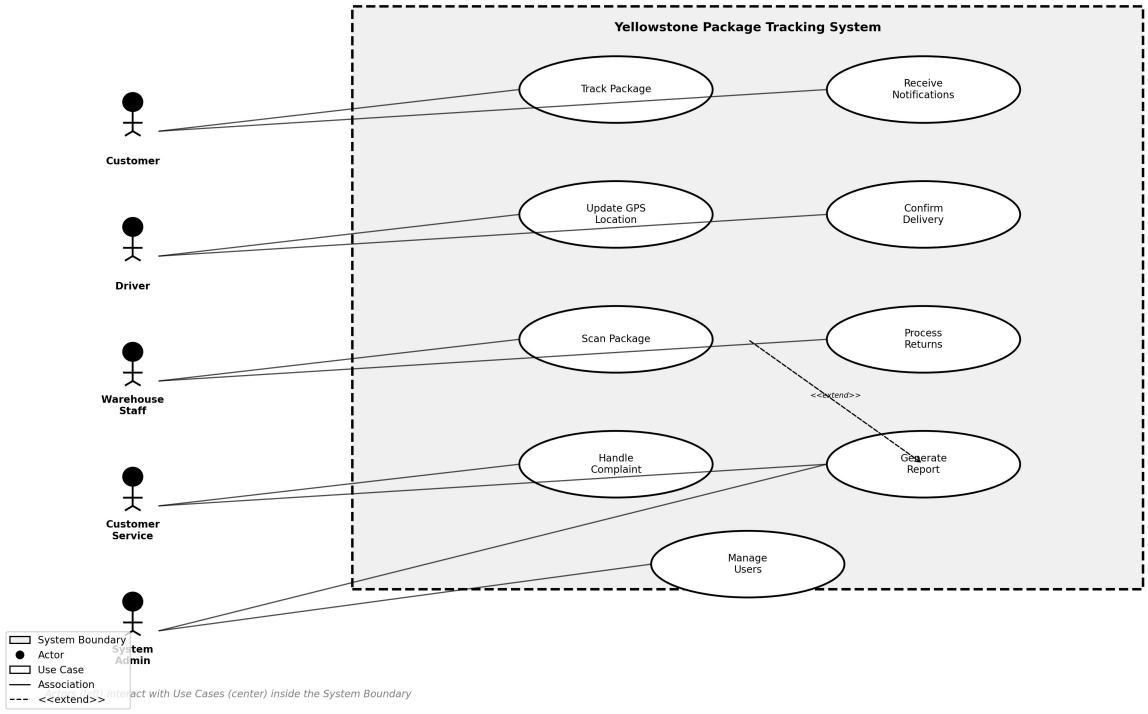
Part A: What is UML and Why is it Important?

Unified Modeling Language (UML) is standardised notation for specifying software systems (Booch et al., 2005). For Yellowstone's tracking system, UML specifies exactly what features the software must have and what each user can do.

Why UML matters: Requirements of clarity prevents misunderstandings. Implementation guides direct developers on what to build. Testing framework: each use case becomes test scenarios. Stakeholder alignment: non-technical managers understand diagrams. Integration: multiple teams work from the same blueprint, preventing incompatible features.

Part B: UML Use Case Diagram for Yellowstone

Figure 3.2: UML Use Case Diagram for Yellowstone Package Tracking System



Part C: Eight Key Use Cases

USE CASE 1: TRACK PACKAGE STATUS Actor: Customer Flow: Customer enters tracking number → System validates → System displays status, location, ETA → Customer views Value: Reduces support calls; Builds confidence

USE CASE 2: RECEIVE NOTIFICATIONS Actor: Customer (passive) Flow: Status changes trigger automatic SMS/Email notifications at each stage (order confirmed, package at warehouse, sorted, in delivery, delivered) Value: Proactive communication reduces "where's my package" calls by 60%; Improves satisfaction

USE CASE 3: UPDATE VEHICLE LOCATION (GPS) Actor: Driver (via GPS device) Flow: GPS records location every 30 seconds → Data sent to server → System stores location → Customer tracking page updates with real-time location Value: Real-time tracking; Better dispatch; Reduces driver-location calls

USE CASE 4: SCAN PACKAGE AT WAREHOUSE Actor: Warehouse Staff Flow: Package arrives → Staff scans barcode → System updates status to "Received at Warehouse" → Customer notified automatically → Package queued for sorting Value: First tracking checkpoint; Accurate inventory; Prevents lost packages

USE CASE 5: PROCESS RETURNS (FAILED DELIVERY) Actor: Warehouse Staff Flow: Failed package returned → Staff scans → System shows original delivery attempt details → Staff inspects condition → System reschedules delivery for next day → Customer notified of new delivery time Value: Second-chance delivery; Maintains satisfaction; Prevents package loss

USE CASE 6: HANDLE COMPLAINTS Actor: Customer Service Agent Flow: Customer reports issue (late, damaged, missing) → Agent pulls up delivery history → Agent categorises complaint → System creates ticket and routes to appropriate department (Compensation, Quality, Missing Items) → Customer receives ticket number and resolution timeline Value: Structured resolution; Root cause analysis; Improves retention

USE CASE 7: GENERATE REPORTS Actor: Manager/Admin Flow: Select report type (daily, weekly, monthly) → Select metrics (on-time %, complaints, avg delivery time, vehicle utilisation) → System aggregates data → System generates report (PDF/Excel) → Manager downloads and reviews Value: Data-driven decisions; Tracks improvement; Shows what's working

USE CASE 8: MANAGE USER ACCOUNTS Actor: System Administrator Flow: Admin accesses user management → Can add new user (system sends welcome email), edit permissions, reset passwords, deactivate users, delete users Value: Security; Role-based access control; Prevents unauthorised access

Part D: System Architecture

Three-tier architecture:

PRESENTATION TIER: Customer website (track packages), Driver mobile app (view deliveries, update status), Warehouse terminal (scan packages), Admin dashboard (view reports, manage users)

APPLICATION TIER: API Gateway (routes all requests), Tracking Service (updates package status), Notification Service (sends SMS/email), Reporting Service (generates reports), User Management (creates/modifies users), Route Optimisation (assigns packages to zones), Complaint Service (creates tickets)

DATA TIER: Main database (packages, users, vehicles, drivers, routes, complaints), Real-time cache Redis (current locations, active deliveries), File storage (photos, invoices, reports)

Part E: Petri Nets vs UML

Petri nets answer: "How should the physical delivery process operate?" They reveal bottlenecks, resource constraints, and inefficiencies. Simulation shows impact of changes: "If we add one vehicle, does sorting still bottleneck?"

UML answers: "What software features do we need?" It specifies user interactions and system capabilities. Use cases define requirements: "System must show real-time location," "System must send notifications when status changes."

Together: Petri nets identify bottlenecks (e.g., sorting is slow). UML specifies software to address it (e.g., dashboard alerts when sorting queue exceeds 100 packages). Physical operations and software work together seamlessly.

REFERENCES

Banks, J., Carson, J.S., Nelson, B.L. and Nicol, D.M., (2014). Discrete-Event System Simulation. 5th edn. Boston: Pearson.

Booch, G., Rumbaugh, J. and Jacobson, I., (2005). The Unified Modeling Language User Guide. 2nd edn. Boston: Addison-Wesley.

Coyle, J.J., Novack, R.A., Gibson, B.J. and Bardi, E.J., (2011). Transportation: A Supply Chain Perspective. 7th edn. Boston: Cengage Learning.

Few, S., (2009). Now You See It: Simple Visualization Techniques for Quantitative Analysis. Oakland: Analytics Press.

Law, A.M., (2015). Simulation Modeling and Analysis. 5th edn. New York: McGraw-Hill Education.

Matsumoto, M. and Nishimura, T., (1998). Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. ACM Transactions on Modeling and Computer Simulation, 8(1), pp.3-30.

Mazza, R. and Bianchi, A.B., (2007). Reading time and task comprehension of information visualizations. International Journal of Human-Computer Studies, 65(12), pp.902-914.

Metropolis, N. and Ulam, S., (1949). The Monte Carlo Method. Journal of the American Statistical Association, 44(247), pp.335-341.

Petri, C.A., (1966). Communication with Automata. PhD Thesis. Bonn: University of Bonn.

Raychaudhuri, S., (2008). Introduction to Monte Carlo simulation. In 2008 Winter Simulation Conference. IEEE. pp.91-100.

Tufte, E.R., (1983). The Visual Display of Quantitative Information. Cheshire: Graphics Press.

Winston, W.L., (2004). Operations Research: Applications and Algorithms. 4th edn. Belmont: Thomson/Brooks-Cole.